INF563 · VÉRIFICATION ET VALIDATION DE LOGICIEL · MASTER 2 SIGL

Devoir 2 – Application To-Do (projets, IndexedDB) testée en BDD

Une application web dont les tâches appartiennent à des **projets** et sont sauvegardées dans **IndexedDB** ; toutes ses fonctionnalités sont décrites en Gherkin et vérifiées dans Chromium par **Cucumber** et **Playwright** ; les tests sont déclenchés à chaque **push** par GitHub Actions.

24 / 24 scénarios BDD réussis, sur 3 exécutions consécutives	**3** features : projets, tâches, persistance IndexedDB	**2** oracles par scénario : l'écran et le contenu réel d'IndexedDB	**Vert** pipeline GitHub Actions au 1er push (3 min 43 s)

Dépôt du devoir

github.com/dradone237/inf563-bdd-todo-indexeddb · branche `main` · 6 commits · rapport Cucumber publié par la CI à chaque exécution.

Enseignant : Evaris Fomekong · Séance de référence : séance 9, « Niveaux, doublures et TDD/BDD », complétée par la séance 12 (« Bout en bout web : Playwright et page objects »)

HTML · CSS · JavaScript · IndexedDB · Java 21 · Maven · Cucumber 7.34.9 · Playwright 1.64.0 · GitHub Actions

Rapport de devoir · 6 novembre 2026

Table des matières

1. Énoncé et compréhension du devoir **3**

1.1 Consignes 3

1.2 Ce que le devoir doit démontrer 3

2. L'application To-Do **3**

2.1 Fonctionnalités 3

2.2 Modification d'une tâche 4

2.3 Modèle de données IndexedDB 4

2.4 Organisation du code 5

3. Démarche BDD **6**

3.1 Des besoins aux features 6

3.2 Un vocabulaire commun 6

4. Architecture des tests **7**

4.1 Arborescence 7

4.2 Vue d'ensemble 7

4.3 Rôle de chaque classe 7

4.4 Bibliothèques 8

5. Les scénarios **8**

5.1 Projets (8 scénarios) 8

5.2 Tâches (12 scénarios) 8

5.3 Persistance IndexedDB (4 scénarios) 9

5.4 Techniques de conception utilisées 9

6. Deux oracles : l'écran et la base **9**

6.1 Pourquoi vérifier IndexedDB directement 9

6.2 Comparer avec le tableau Gherkin 10

7. Isolation et stabilité **10**

8. Défaut détecté par les tests **10**

9. Intégration continue : le push déclenche les tests **11**

10. Résultats **12**

10.1 Exécutions locales 12

10.2 Historique Git 12

11. Lacunes de spécification et décisions **12**

12. Limites et pistes d'amélioration **13**

13. Conclusion **13**

Annexe A – 01_projets.feature **14**

Annexe B – 02_taches.feature **15**

Annexe C – 03_persistance_indexeddb.feature **16**

Annexe D – Accès à IndexedDB (js/db.js) **17**

Annexe E – Pipeline .github/workflows/ci.yml **19**

Annexe F – Commandes utiles **20**

1. Énoncé et compréhension du devoir

1.1 Consignes

- réaliser une application **To-Do** dans laquelle on peut **ajouter** une tâche et **modifier** une tâche ;
- les tâches **appartiennent à des projets** ;
- les données sont **sauvegardées dans IndexedDB**, la base de données intégrée au navigateur ;
- **toutes les fonctionnalités** sont testées en **BDD**, dans le navigateur ;
- un **push** sur le dépôt **déclenche les tests**.

Le devoir porte sur la séance 9 du cours, « Niveaux, doublures et TDD/BDD » (BDD comme spécification exécutable, Gherkin et Cucumber, pipeline GitHub Actions), et sur la séance 12 pour les tests de bout en bout web (Playwright, page objects).

1.2 Ce que le devoir doit démontrer

- décrire chaque fonctionnalité de l'application en **langage métier** (Gherkin, en français) ;
- exécuter ces descriptions dans un **vrai navigateur**, comme le ferait un utilisateur ;
- vérifier que les données sont **réellement sauvegardées** dans IndexedDB, et pas seulement affichées ;
- obtenir des tests **isolés et stables** (F.I.R.S.T.), exécutés automatiquement à chaque push.

2. L'application To-Do

2.1 Fonctionnalités

- **Projets** : créer un projet (nom obligatoire, doublons refusés sans tenir compte de la casse ni des espaces), le supprimer avec ses tâches, filtrer les tâches par projet ;
- **Tâches** : ajouter une tâche dans un projet (titre et projet obligatoires), la **modifier** (titre et/ou projet), la terminer ou la réouvrir, la supprimer ;
- **Compteur** : « *n* tâches, dont *k* terminées », ou « Aucune tâche » ;
- **Persistance** : tout est enregistré dans IndexedDB et retrouvé après un rechargement ou dans un nouvel onglet.

L'application ne dépend d'aucune bibliothèque : HTML, CSS et JavaScript standard (`src/main/webapp`).

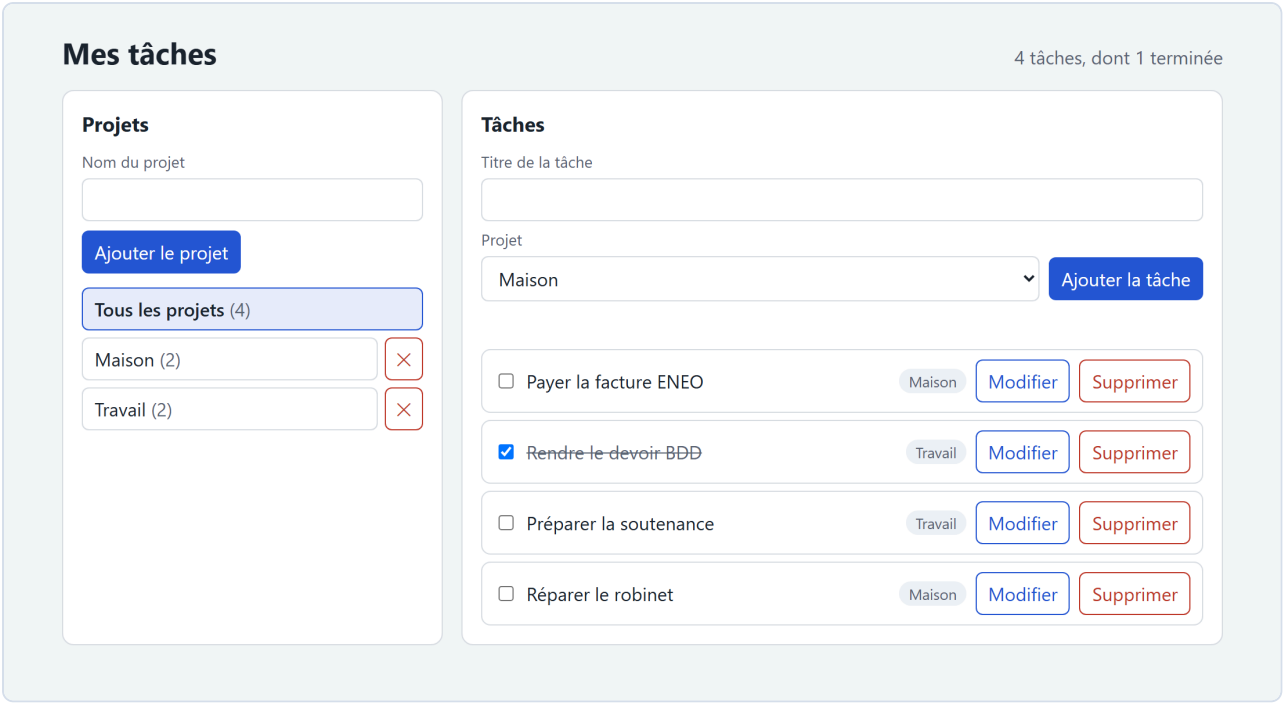


Figure 1 – L'application : deux projets, quatre tâches dont une terminée (barrée), compteur en haut à droite (capture réalisée par Playwright).

2.2 Modification d'une tâche

Le bouton « Modifier » remplace la ligne par un formulaire : nouveau titre, nouveau projet, « Enregistrer » ou « Annuler ». Un titre vidé est refusé, et l'annulation ne change rien : ces deux cas ont leur scénario.

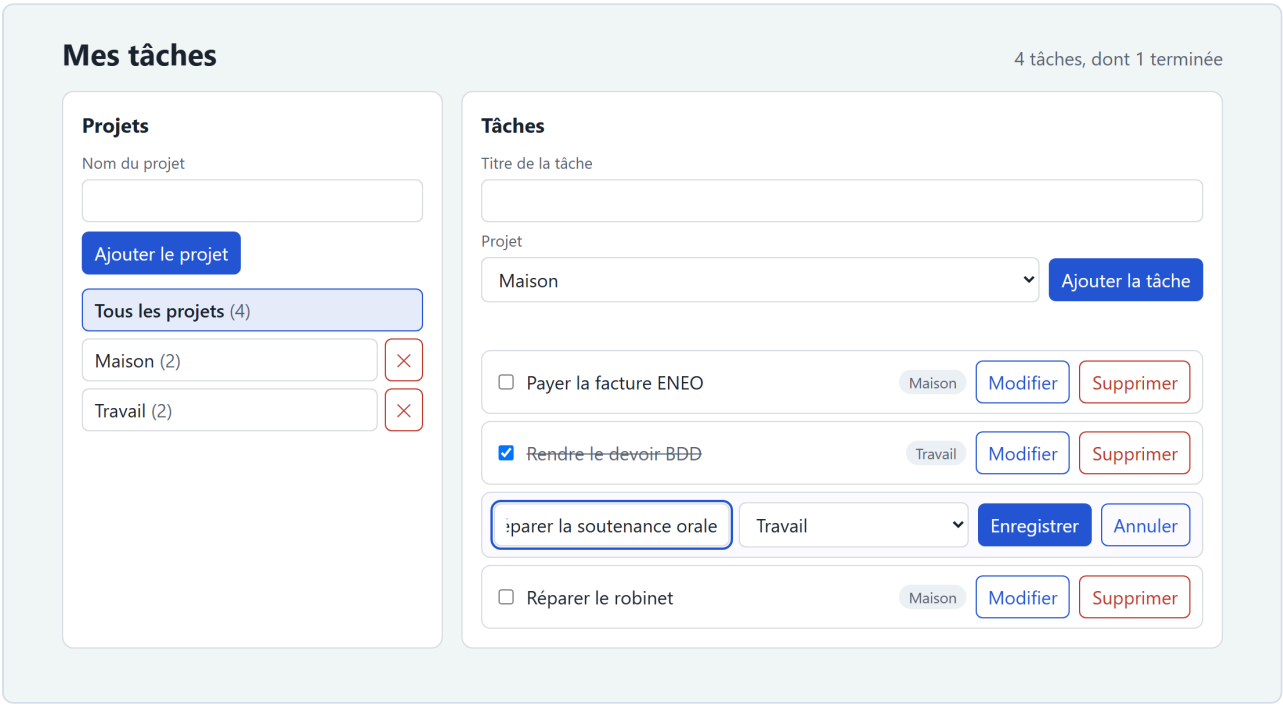


Figure 2 – Modification en cours : la troisième tâche est en mode édition (titre et projet modifiables).

2.3 Modèle de données IndexedDB

La base `todo-inf563` (version 1) contient deux magasins d'objets (*object stores*) :

Magasin	Champs	Index

`projets`	`id` (clé auto-incrémentée), `nom`, `nomNormalise`, `creeLe`	`nomNormalise` **unique** : la base elle-même refuse un doublon
`taches`	`id` (clé auto-incrémentée), `titre`, `projetId`, `terminee`, `creeLe`, `modifieLe`	`projetId` : retrouver les tâches d'un projet

Contenu réel de la base au moment de la figure 1, lu directement dans le navigateur :

id	nom	nomNormalise
1	Maison	`maison`
2	Travail	`travail`

id	titre	projetId	terminee
1	Payer la facture ENEO	1	`false`
2	Rendre le devoir BDD	2	`true`
3	Préparer la soutenance	2	`false`
4	Réparer le robinet	1	`false`

2.4 Organisation du code

Fichier	Rôle
`js/db.js`	Seul accès à IndexedDB : ouverture et création du schéma, lecture, ajout, modification, suppression. Chaque écriture attend la fin de sa transaction.
`js/app.js`	Interface : validations, affichage, événements. Après chaque écriture, l'écran est **reconstruit à partir de la base** : ce qui est affiché est ce qui est sauvegardé.

Trois choix de conception rendent l'application plus sûre et plus facile à tester :

- **Suppression d'un projet en une seule transaction** sur les deux magasins : le projet et ses tâches disparaissent ensemble ou pas du tout, sans tâche orpheline ;
- **textes insérés avec `textContent`**, jamais `innerHTML` : un titre contenant du HTML est affiché tel quel, sans injection (un scénario le vérifie) ;
- **signal de disponibilité** : `data-pret="true"` est posé sur `body` quand la base est ouverte et l'écran rendu ; les tests l'attendent au lieu d'une pause fixe.

```
/** Supprime le projet et toutes ses tâches dans une seule transaction (tout ou rien). */
async supprimerProjet(id) {
  const tx = db.transaction(['projets', 'taches'], 'readwrite');
  tx.objectStore('projets').delete(id);
  const curseur = tx.objectStore('taches').index('projetId').openCursor(IDBKeyRange.only(id));
  curseur.onsuccess = () => {
    if (curseur.result) {
      curseur.result.delete();
      curseur.result.continue();
    }
  };
  await terminee(tx);
},
```

3. Démarche BDD

3.1 Des besoins aux features

Chaque groupe de besoins devient une feature, avec sa user story (« Afin de… En tant que… Je veux… ») et ses scénarios concrets au format **Soit / Quand / Alors** (l'équivalent de Arrange / Act / Assert) :

Feature	User story	Scénarios
`01_projets.feature`	Afin de séparer mes activités, je veux créer des projets auxquels mes tâches appartiennent.	8
`02_taches.feature`	Afin de ne rien oublier, je veux ajouter, modifier, terminer et supprimer des tâches rattachées à un projet.	12
`03_persistance_indexeddb.feature`	Afin de retrouver mon travail, je veux que projets et tâches soient enregistrés dans le navigateur.	4

3.2 Un vocabulaire commun

Les phrases sont réutilisées d'une feature à l'autre (« le projet "Maison" existe », « la tâche "…" existe dans le projet "…" »). Une même phrase correspond toujours à une seule définition d'étape : le Gherkin forme un vocabulaire métier cohérent, et ajouter un scénario ne demande en général aucun nouveau code Java.

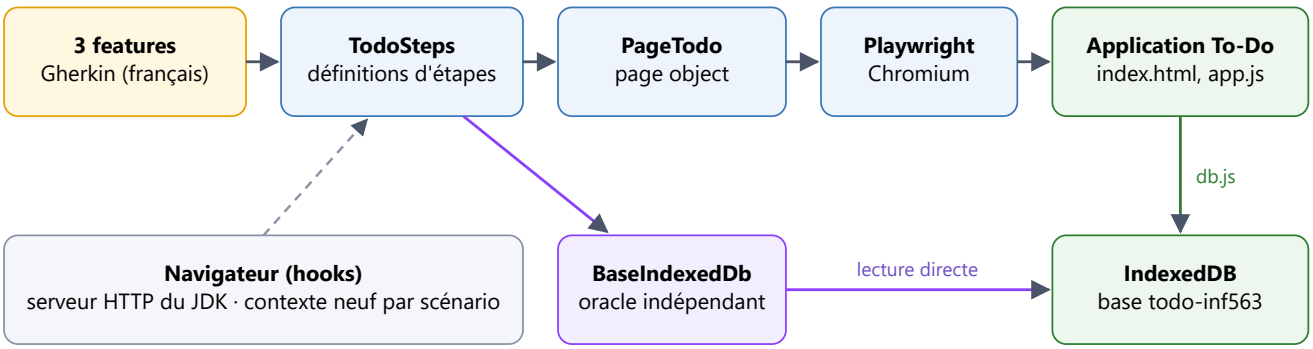
```
Scénario: Supprimer un projet supprime aussi ses tâches
  Soit le projet "Maison" existe
  Et le projet "Travail" existe
  Et la tâche "Faire les courses" existe dans le projet "Maison"
  Et la tâche "Écrire le rapport" existe dans le projet "Travail"
  Quand je supprime le projet "Maison"
  Alors le projet "Maison" n'apparaît plus dans la liste des projets
  Et je vois uniquement les tâches :
    | Écrire le rapport |
  Et la base IndexedDB contient exactement les tâches :
    | tâche             | projet  | terminée |
    | Écrire le rapport | Travail | non      |
```

4. Architecture des tests

4.1 Arborescence

```
inf563-bdd-todo-indexeddb/
├── .github/workflows/ci.yml          pipeline : push → tests
├── pom.xml                           Java 21, JUnit 5, Cucumber, Playwright
└── src/
    ├── main/webapp/                  l'application (sans dépendance)
    │   ├── index.html, css/style.css
    │   └── js/db.js, js/app.js
    └── test/
        ├── java/cm/inf563/todo/
        │   ├── RunCucumberTest.java       point d'entrée JUnit 5 de Cucumber
        │   ├── pages/PageTodo.java        page object
        │   ├── steps/TodoSteps.java       phrases → actions et assertions
        │   ├── support/Navigateur.java    serveur HTTP, Chromium, hooks
        │   └── support/BaseIndexedDb.java oracle : lit/écrit IndexedDB directement
        └── resources/features/
            ├── 01_projets.feature
            ├── 02_taches.feature
            └── 03_persistance_indexeddb.feature
```

4.2 Vue d'ensemble



Bleu : chemin d'un scénario à travers l'interface · violet : vérification directe de la base · pointillés : support

Figure 3 – Architecture des tests : chaque scénario agit sur l'interface au travers du page object, puis vérifie à la fois l'écran et le contenu réel de la base IndexedDB.

4.3 Rôle de chaque classe

Classe	Rôle
`Navigateur`	Démarre le serveur HTTP du JDK (`SimpleFileServer`) sur un port libre pour servir `src/main/webapp` (IndexedDB exige une vraie origine `http://`), un Chromium pour toute la suite et un contexte neuf par scénario ; joint une capture en cas d'échec ; fait échouer le scénario si la page lève une erreur JavaScript.
`PageTodo`	Page object : libellés de formulaire, rôles ARIA et `data-testid` ; méthodes métier (`creerProjet` , `ajouterTache` , `commencerModification` …).
`TodoSteps`	Définitions d'étapes ; conversion des tableaux Gherkin (`DataTable`) en objets Java.
`BaseIndexedDb`	Oracle indépendant : lit et pré-remplit IndexedDB par l'API du navigateur, sans passer par `db.js` .

`RunCucumberTest`	Suite JUnit 5 qui lance Cucumber et produit les rapports HTML et JUnit XML.

4.4 Bibliothèques

Bibliothèque	Version	Usage
Java (Temurin)	21	langage ; `SimpleFileServer` (serveur HTTP intégré au JDK depuis Java 18)
JUnit 5 (Jupiter + Platform Suite)	5.13.4	lancement de Cucumber, assertions sur le contenu de la base
Cucumber JVM (java, junit-platform-engine, picocontainer)	7.34.9	exécution du Gherkin, injection du `Navigateur` dans les étapes
Playwright pour Java	1.64.0	pilotage de Chromium, assertions qui attendent automatiquement

5. Les scénarios

5.1 Projets (8 scénarios)

Scénario	Technique	Oracle
Créer un projet	nominal	projet affiché ; base = [Mémoire M2]
Refuser un projet sans nom (« »)	classe invalide	message « Le nom du projet est obligatoire. » ; aucun projet en base
Plan : doublon – même nom, casse différente, majuscules (3 exemples)	classes d'équivalence	message « Le projet « Maison » existe déjà. » ; base = [Maison]
Doublon saisi avec des espaces autour	valeur limite	même message ; un seul projet en base
Filtrer les tâches par projet	nominal	liste exacte des titres, puis retour à « tous les projets »
Supprimer un projet supprime aussi ses tâches	règle métier	projet et tâches absents de l'écran et de la base ; l'autre projet intact

5.2 Tâches (12 scénarios)

Scénario	Technique	Oracle
Ajouter une tâche dans un projet	nominal	tâche affichée avec son projet ; compteur « 1 tâche, dont 0 terminée » ; base
Plan : tâche invalide – titre vide, sans projet, ni l'un ni l'autre (3 exemples)	classes invalides	message attendu ; aucune tâche en base
Titre fait uniquement d'espaces	valeur limite	« Le titre de la tâche est obligatoire. » ; aucune tâche en base
Modifier le titre d'une tâche	nominal	nouveau titre affiché, ancien absent ; base
Déplacer une tâche vers un autre projet	nominal	nouveau projet affiché ; base

Une modification avec un titre vide est refusée	classe invalide	message ; base inchangée
Annuler une modification ne change rien	nominal	titre d'origine affiché, titre saisi absent
Marquer une tâche comme terminée puis la réouvrir	transition d'état	case et attribut `data-terminee` ; compteur ; base `terminee = true` ; puis retour à l'état initial
Supprimer une tâche	nominal	liste exacte ; base
Un titre contenant du HTML est affiché tel quel	sécurité (injection)	le texte `<b>urgent</b> & important` apparaît littéralement

5.3 Persistance IndexedDB (4 scénarios)

Scénario	Oracle
Une nouvelle base est vide	liste vide, compteur « Aucune tâche », aucun projet ni tâche en base
Les données survivent au rechargement de la page	après `reload`, la tâche est affichée dans son projet et toujours terminée
Les données survivent à la fermeture de l'onglet	onglet fermé puis nouvel onglet du même profil : la tâche est affichée
L'application affiche les données déjà présentes dans IndexedDB	données insérées **directement** dans la base, puis affichées par l'application ; compteur « 3 tâches, dont 1 terminée »

5.4 Techniques de conception utilisées

Les scénarios ne sont pas des exemples pris au hasard : ils viennent des techniques en boîte noire du cours.

- **Classes d'équivalence** : nom de projet valide, vide, en double (casse identique ou différente) ; tâche valide, sans titre, sans projet ;
- **valeurs limites** : chaîne faite uniquement d'espaces, nom entouré d'espaces ;
- **transitions d'état** : tâche à faire → terminée → à faire ; ligne affichée → en édition → enregistrée ou annulée ;
- **plans de scénario** (*Scenario Outline*) : une description, plusieurs exemples, comme `@ParameterizedTest` pour JUnit.

6. Deux oracles : l'écran et la base

6.1 Pourquoi vérifier IndexedDB directement

Vérifier uniquement l'écran ne prouve pas la sauvegarde : une application pourrait afficher une tâche gardée en mémoire sans jamais l'écrire. La classe `BaseIndexedDb` exécute donc dans la page, avec `page.evaluate`, du JavaScript qui ouvre la base par l'API standard du navigateur. Elle **ne réutilise pas** `db.js` : un défaut de l'application ne peut pas se retrouver dans l'oracle.

```
// JavaScript exécuté dans la page par page.evaluate(LIRE) ; il n'utilise pas js/db.js
async () => {
  const db = await ouvrir();                    // indexedDB.open('todo-inf563')
  const lire = (store) => new Promise((ok, ko) => {
    const r = db.transaction(store).objectStore(store).getAll();
    r.onsuccess = () => ok(r.result);
    r.onerror = () => ko(r.error);
  });
  const projets = await lire('projets');
  const taches = await lire('taches');
  db.close();
  return {
    projets: projets.map(p => p.nom),
    taches: taches.map(t => ({
      titre: t.titre,
      projet: (projets.find(p => p.id === t.projetId) || { nom: '(projet introuvable)' }).nom,
      terminee: t.terminee === true
    }))
  };
}
```

6.2 Comparer avec le tableau Gherkin

L'étape « la base IndexedDB contient exactement les tâches » convertit le tableau du scénario en liste d'enregistrements Java (`record TacheStockee`) et la compare, **dans l'ordre**, avec la lecture de la base. La colonne « terminée » n'accepte que « oui » ou « non » : une faute de frappe dans le Gherkin provoque une erreur explicite au lieu d'un faux résultat.

```
@Alors("la base IndexedDB contient exactement les tâches :")
public void tachesEnBase(DataTable tableau) {
    assertEquals(tachesDuTableau(tableau), base.taches());
}
```

7. Isolation et stabilité

- **Un contexte de navigation neuf par scénario** : chaque contexte a son propre stockage, donc IndexedDB est vide au début de chaque scénario. Les scénarios sont indépendants et exécutables dans n'importe quel ordre ;
- **serveur sur un port libre** (port 0) : pas de conflit avec un autre programme, ni en local ni en CI ;
- **aucune pause fixe** : les assertions Playwright attendent la condition, et l'ouverture de l'application attend le signal `data-pret` ;
- **erreurs JavaScript surveillées** : toute exception non interceptée dans la page fait échouer le scénario ;
- **capture d'écran jointe au rapport** en cas d'échec ;
- **stabilité vérifiée** : la suite complète a été exécutée trois fois de suite, avec 24 scénarios sur 24 réussis à chaque fois.

8. Défaut détecté par les tests

À la première exécution, 23 scénarios sur 24 passaient. « Les données survivent au rechargement de la page » échouait : après le rechargement, la tâche cochée n'était plus terminée (`Expected: true, Received: false`).

Cause. L'étape « je coche la tâche » vérifiait que la case était cochée. Or le navigateur coche la case **immédiatement**, **avant** que l'écriture dans IndexedDB soit terminée. L'étape suivante rechargeait la page pendant l'écriture, qui était alors abandonnée. La case cochée n'était pas une preuve de sauvegarde.

Correction. L'attribut `data-terminee` de la ligne n'est rendu qu'**après** la relecture de la base. L'étape attend désormais cet attribut, qui ne peut changer qu'une fois la donnée enregistrée :

```java
// Avant (commit e23f93a)
@Quand("je coche la tâche {string}")
public void jeCocheLaTache(String titre) {
    assertThat(todo.caseTerminee(titre))
        .not().isChecked();
    todo.basculerTerminee(titre);
    assertThat(todo.caseTerminee(titre))
        .isChecked();
}
```

```java
// Après (commit 3cbbe1c)
@Quand("je coche la tâche {string}")
public void jeCocheLaTache(String titre) {
    assertThat(todo.tache(titre))
        .hasAttribute("data-terminee", "false");
    todo.basculerTerminee(titre);
    assertThat(todo.tache(titre))
        .hasAttribute("data-terminee", "true");
}
```

> Le test BDD a révélé une hypothèse fausse : « ce qui est affiché est sauvegardé ». C'est vrai pour la ligne reconstruite depuis la base, mais pas pour l'état d'une case que l'utilisateur vient de cliquer. Le correctif et l'échec qui l'a motivé sont tous deux visibles dans l'historique Git.

9. Intégration continue : le push déclenche les tests

Le fichier `.github/workflows/ci.yml` (annexe E) se déclenche à chaque `push` sur n'importe quelle branche, à chaque *pull request* et à la demande. Il enchaîne :

1. `actions/checkout` puis `actions/setup-java` : JDK Temurin 21 et cache Maven ;
2. installation de Chromium et de ses dépendances système par le CLI Playwright de la **même version** que la bibliothèque Java ;
3. `mvn -B verify` : les 24 scénarios ;
4. publication des rapports Cucumber et Surefire **même en cas d'échec** (`if: always()`).

Le premier push sur `main` (commit `b49fb13`) a déclenché l'exécution n° 1 : **succès** en 3 min 43 s, artefact `rapports-tests` publié.

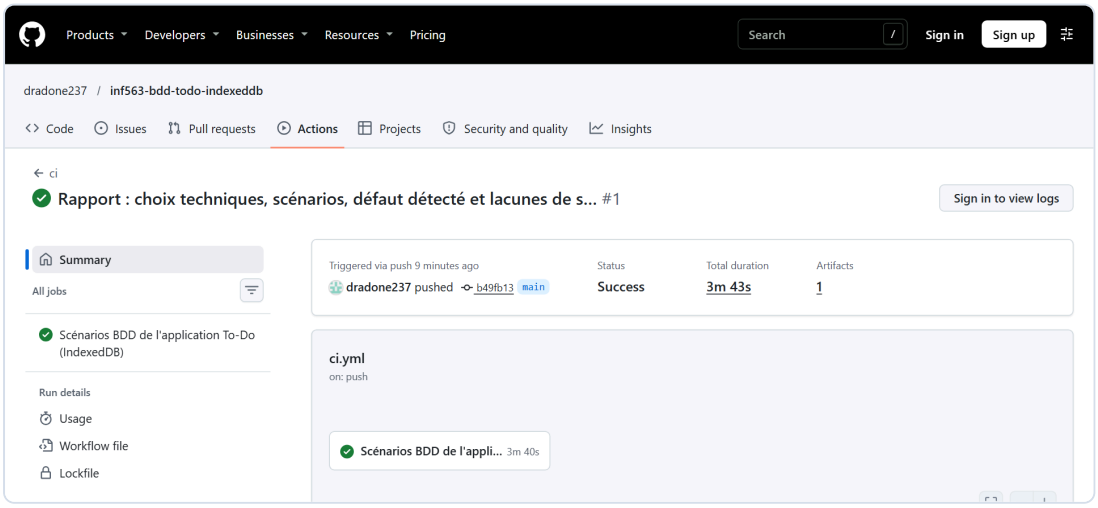


Figure 4 – Exécution n° 1 du pipeline, déclenchée par le push : statut « Success » (github.com/dradone237/inf563-bdd-todo-indexeddb/actions/runs/42291248621).

10. Résultats

10.1 Exécutions locales

Feature	Scénarios	Exécution 1	Exécution 2	Exécution 3
`01_projets.feature`	8	**8 / 8**	**8 / 8**	**8 / 8**
`02_taches.feature`	12	**12 / 12**	**12 / 12**	**12 / 12**
`03_persistance_indexeddb.feature`	4	**4 / 4**	**4 / 4**	**4 / 4**
Total (durée)	**24**	**24 / 24 (12,6 s)**	**24 / 24 (12,3 s)**	**24 / 24 (13,7 s)**

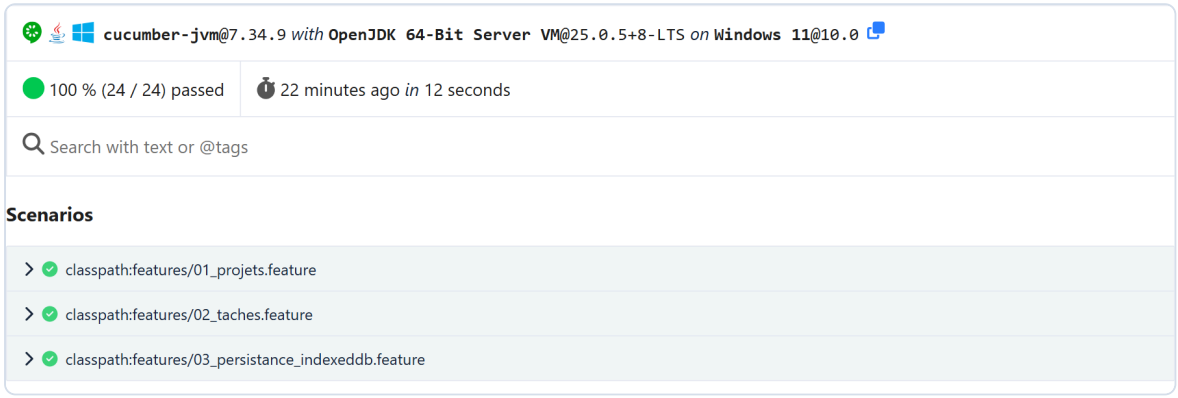


Figure 5 – Rapport HTML produit par Cucumber : 100 % (24 / 24) des scénarios réussis.

10.2 Historique Git

Commit	Message
`6a668a6`	Projet Maven : Java 21, JUnit 5, Cucumber et Playwright
`541a178`	Application To-Do : projets et tâches sauvegardés dans IndexedDB
`e23f93a`	BDD : scénarios Gherkin (projets, tâches, persistance) et étapes Playwright – 23 / 24 au premier essai
`3cbbe1c`	Correctif : synchroniser « je coche la tâche » sur la sauvegarde IndexedDB
`eeabc9a`	CI : GitHub Actions lance les tests à chaque push
`b49fb13`	Rapport : choix techniques, scénarios, défaut détecté et lacunes de spécification

11. Lacunes de spécification et décisions

Question non tranchée par l'énoncé	Décision
« Maison » et « maison » sont-ils deux projets différents ?	Non : comparaison sans tenir compte de la casse ni des espaces autour (index unique `nomNormalise`).
Que deviennent les tâches d'un projet supprimé ?	Elles sont supprimées avec lui, dans la même transaction (sinon tâches orphelines).
Un titre fait uniquement d'espaces est-il valide ?	Non : refusé à l'ajout comme à la modification.

Une tâche peut-elle exister sans projet ?	Non : le projet est obligatoire.
Si le titre et le projet manquent tous les deux, quel message afficher ?	Celui du titre : les règles sont vérifiées dans l'ordre des champs du formulaire.
Dans quel ordre afficher les tâches ?	Ordre de création (clé auto-incrémentée).
Faut-il confirmer une suppression ?	Non : pas de boîte `confirm()`, qui bloquerait l'automatisation ; la suppression est immédiate.

12. Limites et pistes d'amélioration

- Les scénarios tournent sous **Chromium** ; Firefox et WebKit, que Playwright prend aussi en charge, pourraient être ajoutés par une matrice GitHub Actions.
- IndexedDB est propre à chaque navigateur et à chaque profil : il n'y a pas de synchronisation entre appareils (hors du périmètre de l'énoncé).
- Pas de migration de schéma : une version 2 de la base devrait gérer la mise à niveau dans `onupgradeneeded`, et un scénario devrait la couvrir.
- Piste : une confirmation de suppression intégrée à la page (et non une boîte `confirm()`) pour éviter les suppressions accidentelles.

13. Conclusion

L'application To-Do répond à l'énoncé : on ajoute et on modifie des tâches rattachées à des projets, et tout est sauvegardé dans IndexedDB. Ses fonctionnalités sont décrites dans 24 scénarios Gherkin en français, issus des techniques de conception du cours (classes d'équivalence, valeurs limites, transitions d'état). Ils sont exécutés dans un vrai navigateur et vérifient à la fois l'écran et le contenu réel de la base. Les tests sont isolés, sans pause fixe, stables sur plusieurs exécutions, et relancés automatiquement à chaque push par GitHub Actions. Ils ont aussi joué leur rôle de filet de sécurité : ils ont révélé un défaut de synchronisation entre l'affichage et la sauvegarde.

Annexe A – 01_projets.feature

```
# language: fr
@projets
Fonctionnalité: Organiser mes tâches par projet
  Afin de séparer mes différentes activités
  En tant qu'utilisateur de l'application To-Do
  Je veux créer des projets auxquels mes tâches appartiennent

  Contexte:
    Soit j'ouvre l'application To-Do avec une base vide

  Scénario: Créer un projet
    Quand je crée le projet "Mémoire M2"
    Alors le projet "Mémoire M2" apparaît dans la liste des projets
    Et la base IndexedDB contient exactement les projets :
      | Mémoire M2 |

  Scénario: Refuser un projet sans nom
    Quand je crée le projet "   "
    Alors le message "Le nom du projet est obligatoire." s'affiche
    Et la base IndexedDB ne contient aucun projet

  Plan du scénario: Refuser un projet en double (<cas>)
    Soit le projet "Maison" existe
    Quand je crée le projet "<nom saisi>"
    Alors le message "Le projet « Maison » existe déjà." s'affiche
    Et la base IndexedDB contient exactement les projets :
      | Maison |

    Exemples:
      | cas                 | nom saisi   |
      | même nom            | Maison      |
      | casse différente    | maison      |
      | majuscules          | MAISON      |

  Scénario: Refuser un projet en double saisi avec des espaces autour
    Soit le projet "Maison" existe
    Quand je crée le projet "  Maison  "
    Alors le message "Le projet « Maison » existe déjà." s'affiche
    Et la base IndexedDB contient exactement les projets :
      | Maison |

  Scénario: Filtrer les tâches par projet
    Soit le projet "Maison" existe
    Et le projet "Travail" existe
    Et la tâche "Faire les courses" existe dans le projet "Maison"
    Et la tâche "Écrire le rapport" existe dans le projet "Travail"
    Et la tâche "Préparer la soutenance" existe dans le projet "Travail"
    Quand je sélectionne le projet "Travail"
    Alors je vois uniquement les tâches :
      | Écrire le rapport      |
      | Préparer la soutenance |
    Quand je sélectionne tous les projets
    Alors je vois uniquement les tâches :
      | Faire les courses      |
      | Écrire le rapport      |
      | Préparer la soutenance |

  Scénario: Supprimer un projet supprime aussi ses tâches
    Soit le projet "Maison" existe
    Et le projet "Travail" existe
    Et la tâche "Faire les courses" existe dans le projet "Maison"
    Et la tâche "Écrire le rapport" existe dans le projet "Travail"
    Quand je supprime le projet "Maison"
    Alors le projet "Maison" n'apparaît plus dans la liste des projets
```

```
    Et je vois uniquement les tâches :
      | Écrire le rapport |
    Et la base IndexedDB contient exactement les tâches :
      | tâche            | projet  | terminée |
      | Écrire le rapport | Travail | non      |
```

Annexe B – 02_taches.feature

```
# language: fr
@taches
Fonctionnalité: Gérer les tâches d'un projet
  Afin de ne rien oublier
  En tant qu'utilisateur de l'application To-Do
  Je veux ajouter, modifier, terminer et supprimer des tâches rattachées à un projet

  Contexte:
    Soit j'ouvre l'application To-Do avec une base vide
    Et le projet "Maison" existe
    Et le projet "Travail" existe

  Scénario: Ajouter une tâche dans un projet
    Quand j'ajoute la tâche "Payer la facture ENEO" dans le projet "Maison"
    Alors la tâche "Payer la facture ENEO" est affichée dans le projet "Maison"
    Et le compteur indique "1 tâche, dont 0 terminée"
    Et la base IndexedDB contient exactement les tâches :
      | tâche                | projet | terminée |
      | Payer la facture ENEO | Maison | non      |

  Plan du scénario: Refuser une tâche invalide (<cas>)
    Quand j'ajoute la tâche "<titre>" dans le projet "<projet>"
    Alors le message "<message>" s'affiche
    Et la base IndexedDB ne contient aucune tâche

    Exemples:
      | cas               | titre           | projet | message                              |
      | titre vide        |                 | Maison | Le titre de la tâche est obligatoire. |
      | sans projet       | Acheter du pain |        | Choisissez un projet pour la tâche.   |
      | ni titre ni projet |                 |        | Le titre de la tâche est obligatoire. |

  Scénario: Refuser une tâche dont le titre ne contient que des espaces
    Quand j'ajoute la tâche "    " dans le projet "Maison"
    Alors le message "Le titre de la tâche est obligatoire." s'affiche
    Et la base IndexedDB ne contient aucune tâche

  Scénario: Modifier le titre d'une tâche
    Soit la tâche "Apeler le plombier" existe dans le projet "Maison"
    Quand je renomme la tâche "Apeler le plombier" en "Appeler le plombier"
    Alors la tâche "Appeler le plombier" est affichée dans le projet "Maison"
    Et la tâche "Apeler le plombier" n'est plus affichée
    Et la base IndexedDB contient exactement les tâches :
      | tâche              | projet | terminée |
      | Appeler le plombier | Maison | non      |

  Scénario: Déplacer une tâche vers un autre projet
    Soit la tâche "Réviser le chapitre 4" existe dans le projet "Maison"
    Quand je déplace la tâche "Réviser le chapitre 4" vers le projet "Travail"
    Alors la tâche "Réviser le chapitre 4" est affichée dans le projet "Travail"
    Et la base IndexedDB contient exactement les tâches :
      | tâche               | projet  | terminée |
      | Réviser le chapitre 4 | Travail | non      |

  Scénario: Une modification avec un titre vide est refusée
    Soit la tâche "Laver la voiture" existe dans le projet "Maison"
```

```
    Quand je renomme la tâche "Laver la voiture" en ""
    Alors le message "Le titre de la tâche est obligatoire." s'affiche
    Et la base IndexedDB contient exactement les tâches :
      | tâche            | projet | terminée |
      | Laver la voiture | Maison | non      |

  Scénario: Annuler une modification ne change rien
    Soit la tâche "Laver la voiture" existe dans le projet "Maison"
    Quand je commence à renommer la tâche "Laver la voiture" en "Autre chose" puis j'annule
    Alors la tâche "Laver la voiture" est affichée dans le projet "Maison"
    Et la tâche "Autre chose" n'est plus affichée

  Scénario: Marquer une tâche comme terminée puis la réouvrir
    Soit la tâche "Rendre le devoir BDD" existe dans le projet "Travail"
    Quand je coche la tâche "Rendre le devoir BDD"
    Alors la tâche "Rendre le devoir BDD" est marquée comme terminée
    Et le compteur indique "1 tâche, dont 1 terminée"
    Et la base IndexedDB contient exactement les tâches :
      | tâche               | projet  | terminée |
      | Rendre le devoir BDD | Travail | oui      |
    Quand je décoche la tâche "Rendre le devoir BDD"
    Alors la tâche "Rendre le devoir BDD" n'est pas marquée comme terminée

  Scénario: Supprimer une tâche
    Soit la tâche "Faire les courses" existe dans le projet "Maison"
    Et la tâche "Ranger le bureau" existe dans le projet "Maison"
    Quand je supprime la tâche "Faire les courses"
    Alors je vois uniquement les tâches :
      | Ranger le bureau |
    Et la base IndexedDB contient exactement les tâches :
      | tâche            | projet | terminée |
      | Ranger le bureau | Maison | non      |

  Scénario: Un titre contenant du HTML est affiché tel quel
    Quand j'ajoute la tâche "<b>urgent</b> & important" dans le projet "Travail"
    Alors la tâche "<b>urgent</b> & important" est affichée dans le projet "Travail"
```

Annexe C – 03_persistance_indexeddb.feature

```
# language: fr
@persistance
Fonctionnalité: Sauvegarde des projets et des tâches dans IndexedDB
  Afin de retrouver mon travail après avoir fermé ou rechargé la page
  En tant qu'utilisateur de l'application To-Do
  Je veux que mes projets et mes tâches soient enregistrés dans le navigateur

  Scénario: Une nouvelle base est vide
    Soit j'ouvre l'application To-Do avec une base vide
    Alors la liste des tâches est vide
    Et le compteur indique "Aucune tâche"
    Et la base IndexedDB ne contient aucun projet
    Et la base IndexedDB ne contient aucune tâche

  Scénario: Les données survivent au rechargement de la page
    Soit j'ouvre l'application To-Do avec une base vide
    Et le projet "Maison" existe
    Et la tâche "Arroser les plantes" existe dans le projet "Maison"
    Et je coche la tâche "Arroser les plantes"
    Quand je recharge la page
    Alors la tâche "Arroser les plantes" est affichée dans le projet "Maison"
    Et la tâche "Arroser les plantes" est marquée comme terminée

  Scénario: Les données survivent à la fermeture de l'onglet
```

```
    Soit j'ouvre l'application To-Do avec une base vide
    Et le projet "Travail" existe
    Et la tâche "Envoyer le compte rendu" existe dans le projet "Travail"
    Quand je ferme l'onglet et j'ouvre l'application dans un nouvel onglet
    Alors la tâche "Envoyer le compte rendu" est affichée dans le projet "Travail"

  Scénario: L'application affiche les données déjà présentes dans IndexedDB
    Soit la base IndexedDB contient déjà :
      | tâche                   | projet  | terminée |
      | Préparer la réunion     | Travail | non      |
      | Envoyer le compte rendu | Travail | oui      |
      | Réparer le robinet      | Maison  | non      |
    Quand j'ouvre l'application To-Do
    Alors je vois uniquement les tâches :
      | Préparer la réunion     |
      | Envoyer le compte rendu |
      | Réparer le robinet      |
    Et la tâche "Envoyer le compte rendu" est marquée comme terminée
    Et le compteur indique "3 tâches, dont 1 terminée"
```

Annexe D – Accès à IndexedDB (js/db.js)

```
/*
 * Accès à IndexedDB : toute la persistance de l'application passe par ce module.
 *
 * Base "todo-inf563", version 1
 *   projets : { id (auto), nom, nomNormalise (index unique), creeLe }
 *   taches  : { id (auto), titre, projetId (index), terminee, creeLe, modifieLe }
 */
const TodoDb = (() => {
  const NOM_BASE = 'todo-inf563';
  const VERSION = 1;

  const normaliser = (texte) => texte.trim().toLocaleLowerCase('fr');
  const maintenant = () => new Date().toISOString();

  function resultat(requete) {
    return new Promise((resoudre, rejeter) => {
      requete.onsuccess = () => resoudre(requete.result);
      requete.onerror = () => rejeter(requete.error);
    });
  }

  function terminee(transaction) {
    return new Promise((resoudre, rejeter) => {
      transaction.oncomplete = () => resoudre();
      transaction.onerror = () => rejeter(transaction.error);
      transaction.onabort = () => rejeter(transaction.error || new Error('Transaction annulée'));
    });
  }

  function creerSchema(db) {
    const projets = db.createObjectStore('projets', { keyPath: 'id', autoIncrement: true });
    projets.createIndex('nomNormalise', 'nomNormalise', { unique: true });
    const taches = db.createObjectStore('taches', { keyPath: 'id', autoIncrement: true });
    taches.createIndex('projetId', 'projetId');
  }

  async function ouvrir() {
    const ouverture = indexedDB.open(NOM_BASE, VERSION);
    ouverture.onupgradeneeded = () => creerSchema(ouverture.result);
    const db = await resultat(ouverture);
```

```
    const lireTout = (store) => resultat(db.transaction(store).objectStore(store).getAll());

    return {
      listerProjets: () => lireTout('projets'),
      listerTaches: () => lireTout('taches'),

      /** Rejette avec une ConstraintError si un projet de même nom (casse ignorée) existe. */
      async ajouterProjet(nom) {
        const tx = db.transaction('projets', 'readwrite');
        const ajout = tx.objectStore('projets').add({ nom, nomNormalise: normaliser(nom), creeLe: maintenant()
});
        await terminee(tx);
        return ajout.result;
      },

      /** Supprime le projet et toutes ses tâches dans une seule transaction (tout ou rien). */
      async supprimerProjet(id) {
        const tx = db.transaction(['projets', 'taches'], 'readwrite');
        tx.objectStore('projets').delete(id);
        const curseur = tx.objectStore('taches').index('projetId').openCursor(IDBKeyRange.only(id));
        curseur.onsuccess = () => {
          if (curseur.result) {
            curseur.result.delete();
            curseur.result.continue();
          }
        };
        await terminee(tx);
      },

      async ajouterTache({ titre, projetId }) {
        const tx = db.transaction('taches', 'readwrite');
        const date = maintenant();
        const ajout = tx.objectStore('taches').add({ titre, projetId, terminee: false, creeLe: date, modifieLe:
date });
        await terminee(tx);
        return ajout.result;
      },

      async modifierTache(id, modifications) {
        const tx = db.transaction('taches', 'readwrite');
        const store = tx.objectStore('taches');
        const lecture = store.get(id);
        lecture.onsuccess = () => {
          if (!lecture.result) {
            tx.abort();
            return;
          }
          store.put({ ...lecture.result, ...modifications, modifieLe: maintenant() });
        };
        await terminee(tx);
      },

      async supprimerTache(id) {
        const tx = db.transaction('taches', 'readwrite');
        tx.objectStore('taches').delete(id);
        await terminee(tx);
      },
    };
  }

  return { ouvrir, normaliser, NOM_BASE };
})();
```

Annexe E – Pipeline .github/workflows/ci.yml

```yaml
# Intégration continue : chaque push (sur n'importe quelle branche) et chaque
# pull request déclenche automatiquement tous les scénarios BDD dans Chromium.
# Un pipeline rouge = on ne fusionne pas (règle de protection de la branche main).
name: ci

on:
  push:
  pull_request:
  workflow_dispatch:   # bouton « Run workflow » pour relancer à la main

jobs:
  tests:
    name: "Scénarios BDD de l'application To-Do (IndexedDB)"
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - name: Installer Chromium et ses dépendances système (Playwright)
        run: >
          mvn -B -q exec:java -e
          -Dexec.mainClass=com.microsoft.playwright.CLI
          -Dexec.args="install --with-deps chromium"
          -Dexec.classpathScope=test

      - name: Scénarios Gherkin (BDD) dans Chromium
        run: mvn -B verify
        env:
          PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD: '1'

      - name: Publier les rapports (même en cas d'échec)
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: rapports-tests
          path: |
            target/cucumber-reports/
            target/surefire-reports/
```

Annexe F – Commandes utiles

```
# Une seule fois : télécharger Chromium pour Playwright
mvn exec:java -e -Dexec.mainClass=com.microsoft.playwright.CLI -Dexec.args="install chromium" -
Dexec.classpathScope=test

mvn verify                                          # les 24 scénarios
mvn verify -Dheadless=false -DslowMo=300            # démonstration : navigateur visible et ralenti
mvn verify -Dcucumber.filter.tags=@persistance      # une seule feature (@projets, @taches, @persistance)

# Utiliser l'application à la main, puis ouvrir http://localhost:8080
jwebserver -d <chemin absolu vers src/main/webapp> -p 8080

# Rapport lisible : target/cucumber-reports/cucumber.html
```